

COMP3026E Distributed Systems

8. View Change & Consensus

James Jianqiao Yu

Harbin Institute of Technology, Shenzhen

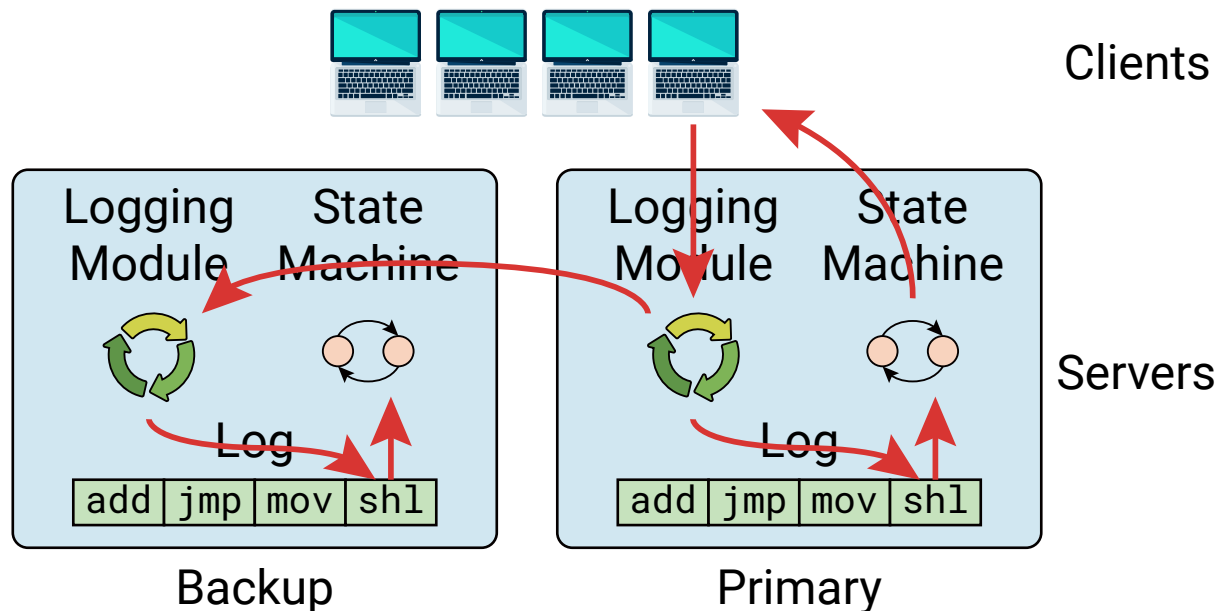
Today



1. From primary-backup to viewstamped replication
2. Consensus

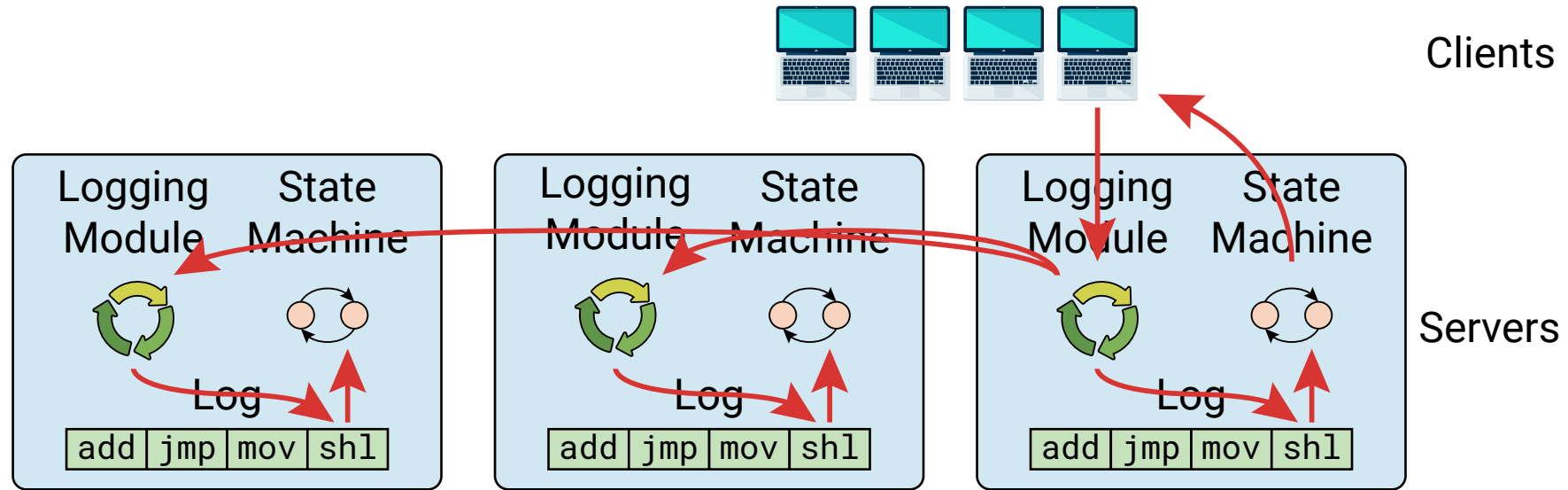


Review: Primary-Backup Replication



- Nominate one replica **primary**
 - Clients send all requests to primary
 - Primary orders clients' requests

From Two to Many Replicas



- Primary-backup with many replicas
 - Primary waits for acknowledgement from all backups

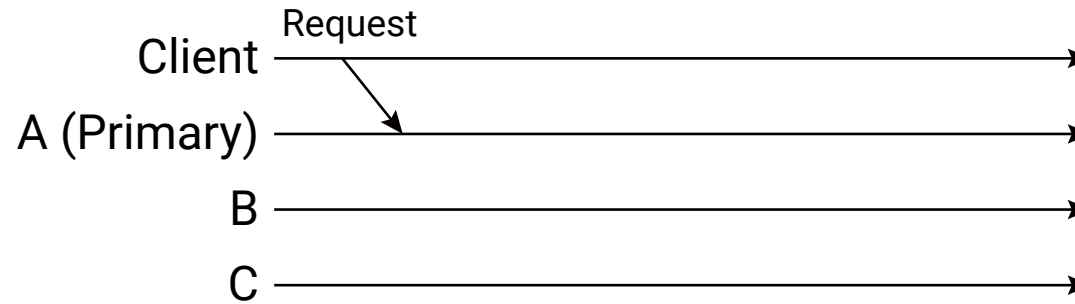
What Else Can We Do With More Replicas?

- Viewstamped Replication:
 - State Machine Replication for any number of replicas
 - **Replica group**: Group of $2f + 1$ replicas
 - Protocol can tolerate f replica crashes
- Differences with primary-backup
 - Don't need to wait for all replicas to reply

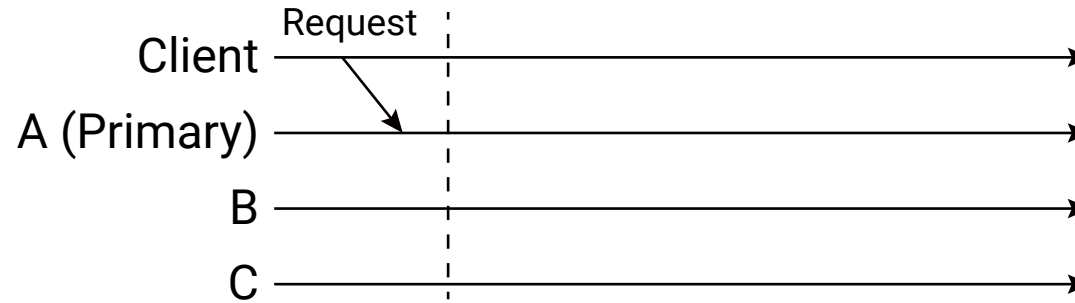
What Else Can We Do With More Replicas?

- Viewstamped Replication:
 - State Machine Replication for any number of replicas
 - **Replica group**: Group of $2f + 1$ replicas
 - Protocol can tolerate f replica crashes
- Differences with primary-backup
 - Don't need to wait for all replicas to reply
- **Replica State**
 1. **Configuration**: identities of all $2f + 1$ replicas
 2. **Log** with clients' requests in assigned order

Normal Operation

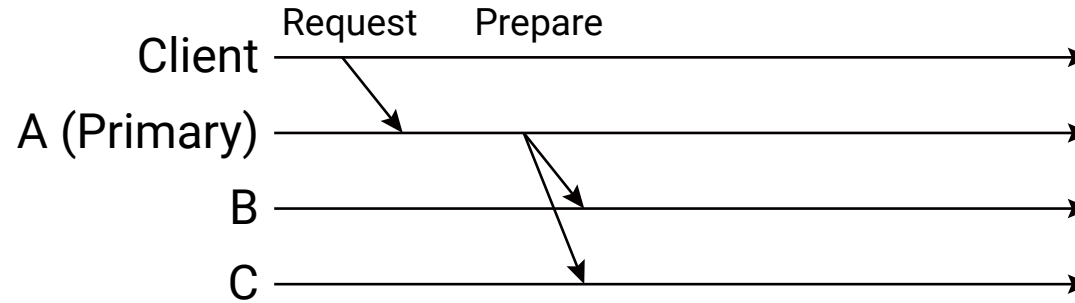


Normal Operation



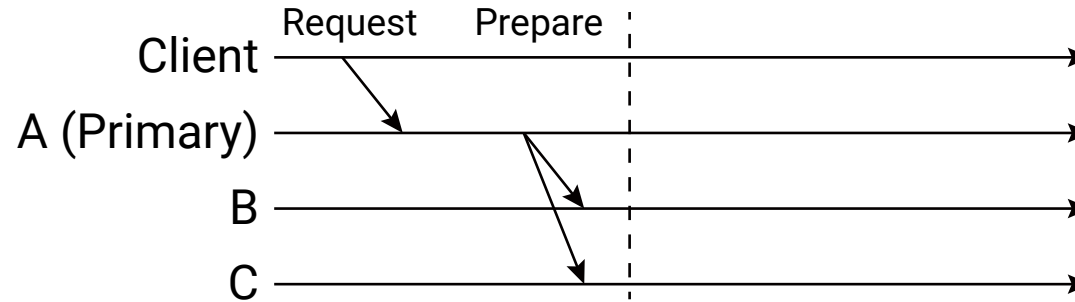
1. Primary adds request to end of its log

Normal Operation



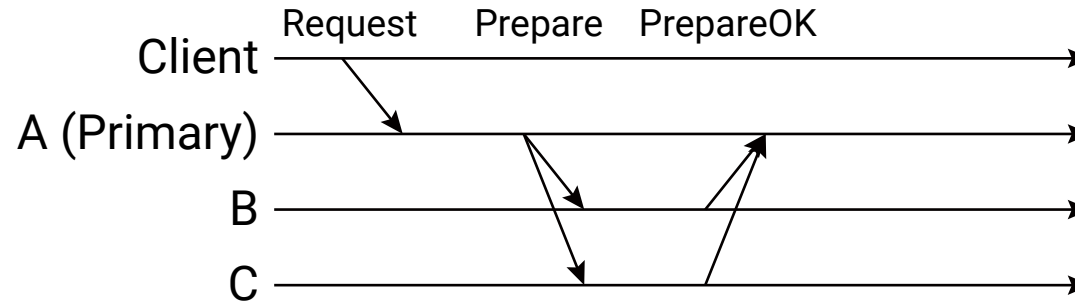
1. Primary adds request to end of its log

Normal Operation



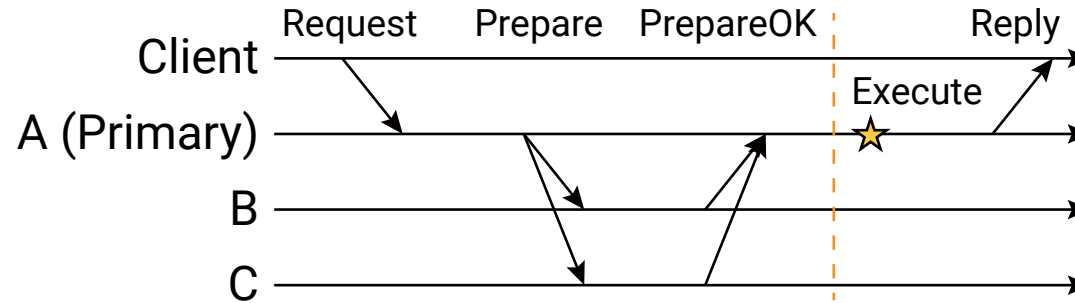
1. Primary adds request to end of its log
2. Replicas add requests to their logs in primary's log order

Normal Operation



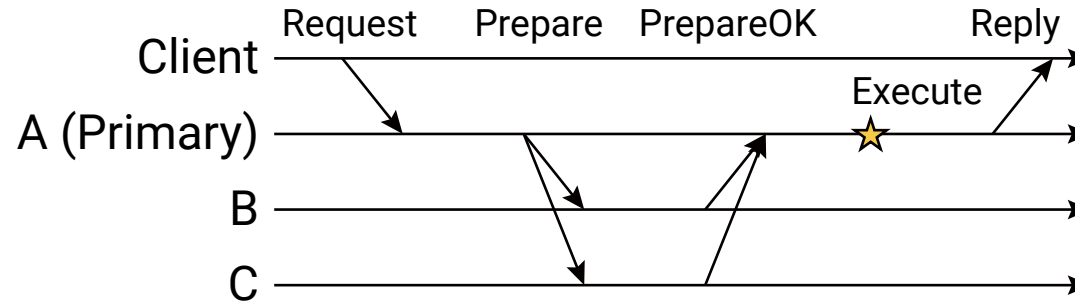
1. Primary adds request to end of its log
2. Replicas add requests to their logs in primary's log order

Normal Operation



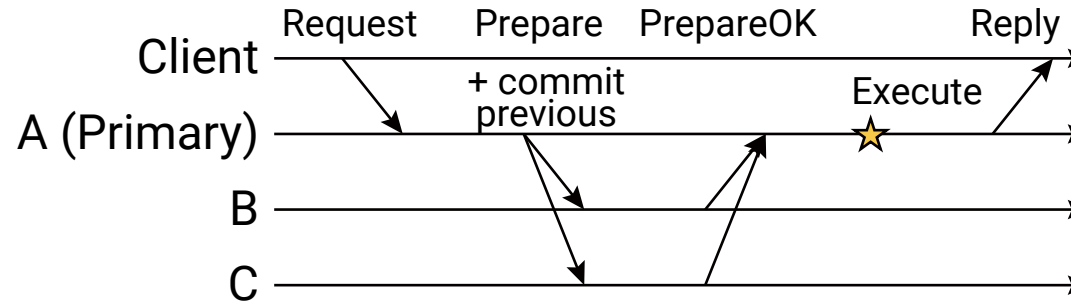
1. Primary adds request to end of its log
2. Replicas add requests to their logs in primary's log order
3. Primary **waits for f PrepareOKs** -> request is **committed**

Normal Operation: Key Points



- Protocol provides state machine replication
- On execute, primary knows request in $f + 1 = 2$ nodes' logs
 - Even if $f = 1$ then crash, ≥ 1 retains request in log

Normal Operation: Key Points



- Previous Request's commit **piggybacked** on current Prepare
- No client Request after a timeout period?
 - Primary sends Commit message to all backups

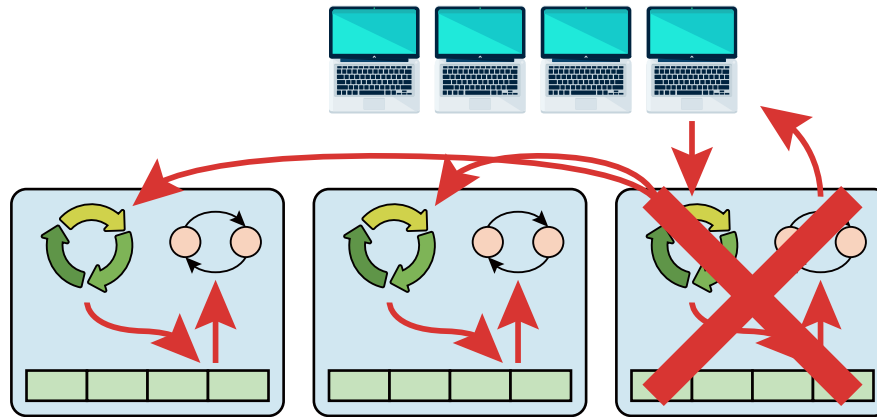
▶ The Need For a View Change



- So far: Works for f failed backup replicas

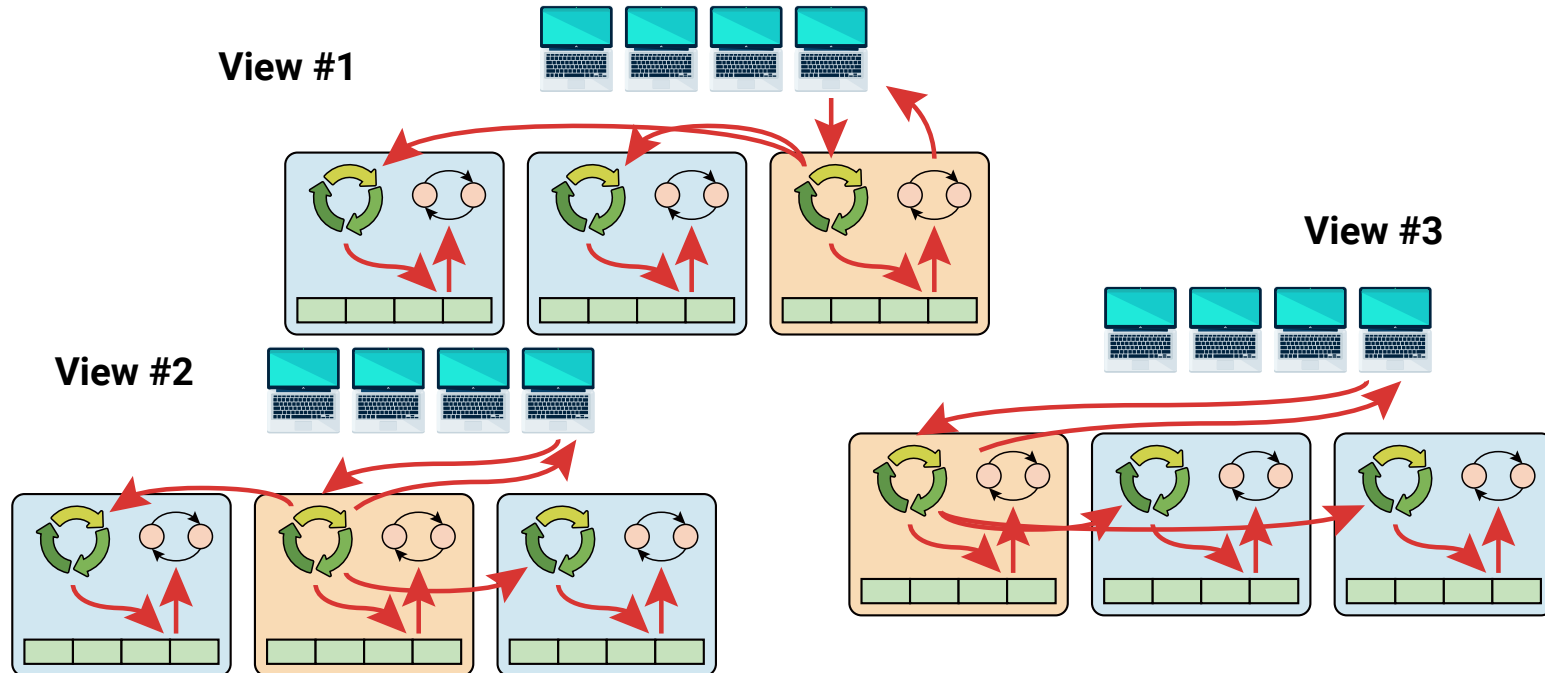
The Need For a View Change

- So far: Works for f failed backup replicas
- But what if the f failures include a failed primary?
 - All clients' requests go to the failed primary
 - System halts



Views

- Let **different replicas** assume role of primary over time
- System moves through a sequence of views
 - View = (view number, primary id, backup id, ...)



Correctly Changing Views



- View changes happen locally at each replica
- Old primary executes requests in the old view, new primary executes requests in the new view

Correctly Changing Views



- View changes happen locally at each replica
- Old primary executes requests in the old view, new primary executes requests in the new view
- Goal of changing views: **Consistency**. Committed requests
 - Survive in the new view
 - Retain the same order in the new view

Correctly Changing Views

- View changes happen locally at each replica
- Old primary executes requests in the old view, new primary executes requests in the new view
- Goal of changing views: **Consistency**. Committed requests
 - Survive in the new view
 - Retain the same order in the new view
- Agreement: How do replicas **agree to move** to a new view?
- Data Gap: How do replicas **agree on what was executed** (and in what order) in the old view?

- Definition:
 - A general agreement about something
 - An idea or opinion that is shared by all the people in a group



- Definition:
 - A general agreement about something
 - An idea or opinion that is shared by all the people in a group
- Group of servers want to:
 - Make sure all servers in group receive the same updates in the same order as each other

- Definition:
 - A general agreement about something
 - An idea or opinion that is shared by all the people in a group
- Group of servers want to:
 - Make sure all servers in group receive the same updates in the same order as each other
 - Maintain own lists (views) on who is a current member of the group, and update lists when somebody leaves/fails

- Definition:
 - A general agreement about something
 - An idea or opinion that is shared by all the people in a group
- Group of servers want to:
 - Make sure all servers in group receive the same updates in the same order as each other
 - Maintain own lists (views) on who is a current member of the group, and update lists when somebody leaves/fails
 - Elect a leader in group, and inform everybody

- Definition:
 - A general agreement about something
 - An idea or opinion that is shared by all the people in a group
- Group of servers want to:
 - Make sure all servers in group receive the same updates in the same order as each other
 - Maintain own lists (views) on who is a current member of the group, and update lists when somebody leaves/fails
 - Elect a leader in group, and inform everybody
 - Ensure mutually exclusive (one process at a time only) access to a critical resource like a file

- Given a set of processors, each with an initial value:
 - **Agreement**: All processes that decide do so on the same value
 - **Validity**: Value decided must have proposed by some process
 - **Termination**: All non-faulty processes eventually decide on a value
- Why Consensus as we already have viewstamped replication?
 - Consensus is hard due to asynchrony and failures
 - Crashes? Delayed messages? FLP Impossibility

► Safety vs. Liveness Properties



- **Safety** (bad things never happen)
- **Liveness** (good things eventually happen)

- **Safety** (bad things never happen)
 - **Agreement**: All processes that decide do so on the same value
 - **Validity**: Value decided must have proposed by some process
- **Liveness** (good things eventually happen)
 - **Termination**: All non-faulty processes eventually decide on a value

Paxos' Safety and Liveness



- Paxos is **always safe**
- Paxos is **very often live** (but not always, more later)
- Also true for Viewstamped Replication, RAFT, and other similar protocols

Roles of a Process in Paxos

- Three conceptual roles
 - **Proposers** propose values
 - **Acceptors** accept values, where value is chosen if majority accept
 - **Learners** learn the outcome (chosen value)
- In reality, a process can play any/all roles

Strawman Example



- 3 proposers, 1 acceptor
 - Acceptor accepts first value received

Strawman Example

- 3 proposers, 1 acceptor
 - Acceptor accepts first value received
 - No liveness with single failure
 - Need multiple acceptors

Strawman Example



- 3 proposers, 1 acceptor
 - Acceptor accepts first value received
 - No liveness with single failure
 - Need multiple acceptors
- 3 proposers, 3 acceptors
 - Accept first value received, learners choose common value known by majority

Strawman Example



- 3 proposers, 1 acceptor
 - Acceptor accepts first value received
 - No liveness with single failure
 - Need multiple acceptors
- 3 proposers, 3 acceptors
 - Accept first value received, learners choose common value known by majority
 - But no such majority is guaranteed
 - Need acceptors to be capable of accepting multiple values

- Each acceptor accepts **multiple proposals**
 - Hopefully one of multiple accepted proposals will have a majority vote (and we determine that)
 - If not, rinse and repeat (more on this)

- Each acceptor accepts **multiple proposals**
 - Hopefully one of multiple accepted proposals will have a majority vote (and we determine that)
 - If not, rinse and repeat (more on this)
- How do we select among multiple proposals?
 - Ordering: proposal is tuple (proposal #, value) = (n, v)
 - Proposal # strictly increasing, globally unique
 - Globally unique?
 - Trick: set low-order bits to proposer's ID

Paxos Protocol Overview

- **Proposers:**
 1. Choose a proposal number n
 2. Ask acceptors if any accepted proposals with $n_a < n$
 3. If existing proposal v_a returned, propose same value (n, v_a)
 4. Otherwise, propose own value (n, v)
 - Note altruism: goal is to reach consensus, **not “win”**
- **Accepters** try to accept value with highest proposal n
- **Learners** are passive and wait for the outcome



Paxos Phase 1



- Proposer:
 - Choose proposal n , send `<prepare, n>` to acceptors

Paxos Phase 1

- Proposer:
 - Choose proposal n , send `<prepare, n>` to acceptors
- Acceptors:
 - If $n > n_h$
 - $n_h = n \leftarrow$ promise not to accept any new proposals $n' < n$
 - If no prior proposal accepted
 - Reply `<promise, n,  $\emptyset$ >`
 - Else
 - Reply `<promise, n, (na, va)>`
 - Else
 - Reply `<prepare-failed>`

Paxos Phase 2

- Proposer:
 - If receive promise from **majority** of acceptors,
 - Determine v_a returned with highest n_a , if exists
 - Send **<accept, (n, va || v)>** to acceptors
- Acceptors:
 - Upon receiving (n, v) , if $n \geq n_h$,
 - Accept proposal and notify learner(s)

Paxos Phase 3



- Learners need to know which value chosen



Paxos Phase 3



- Learners need to know which value chosen
- Approach #1
 - Each acceptor notifies all learners
 - More expensive

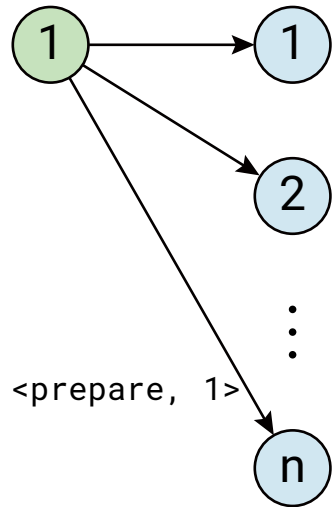


Paxos Phase 3

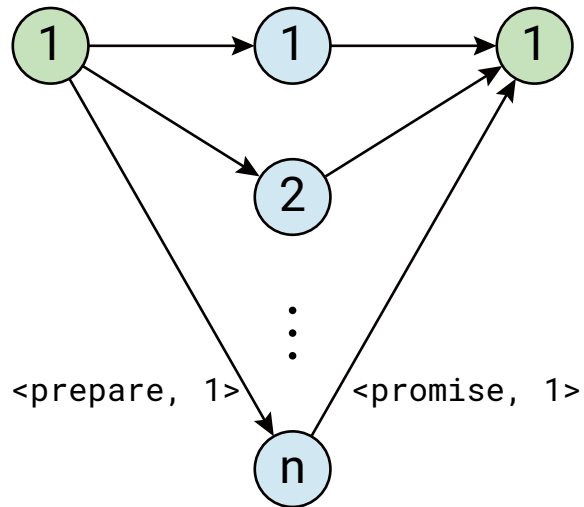


- Learners need to know which value chosen
- Approach #1
 - Each acceptor notifies all learners
 - More expensive
- Approach #2
 - Elect a “distinguished learner”
 - Acceptors notify elected learner, which informs others
 - Failure-prone

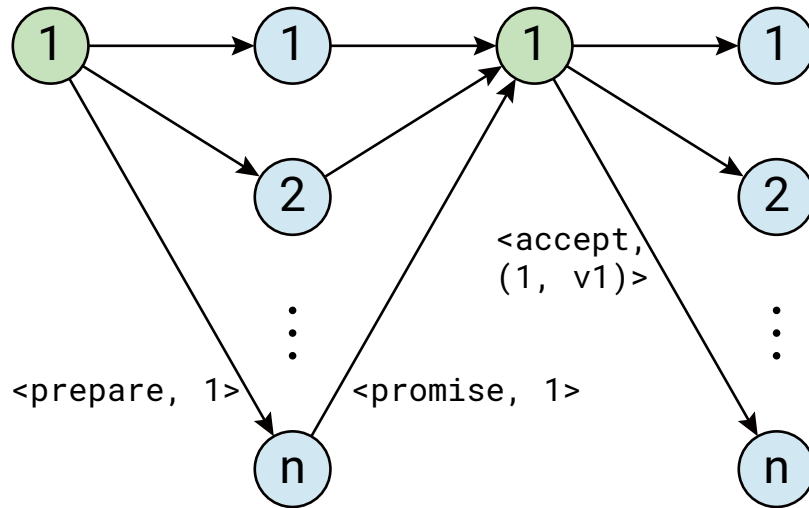
Paxos: Well-behaved Run



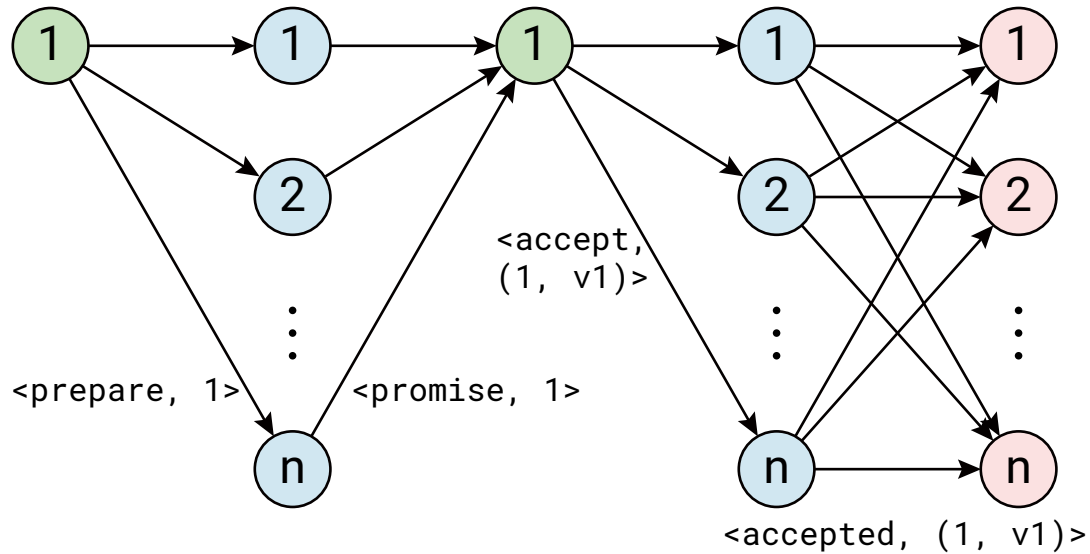
Paxos: Well-behaved Run



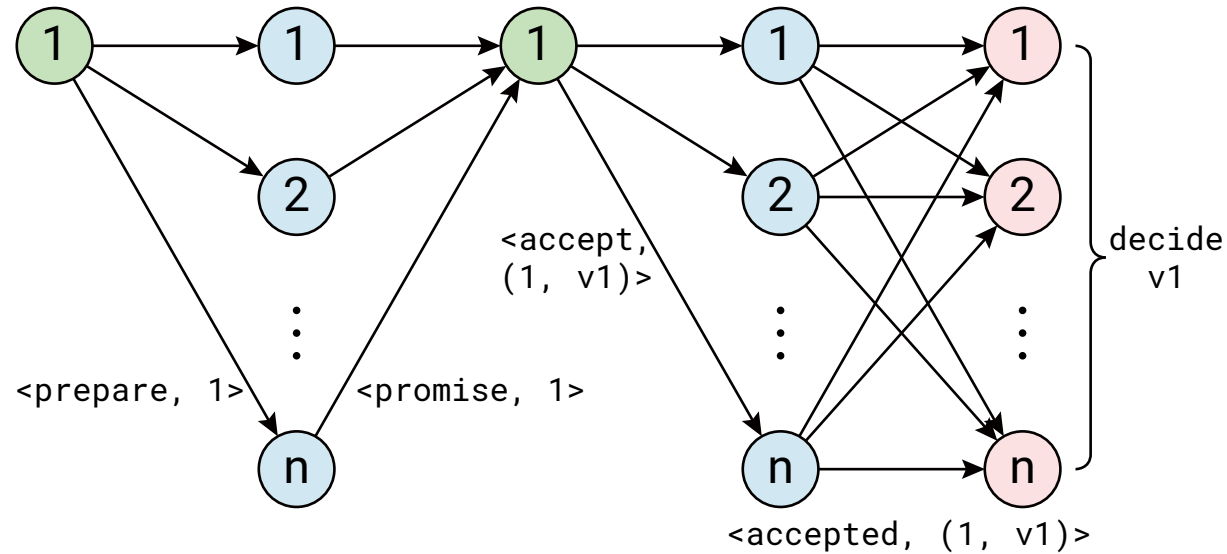
Paxos: Well-behaved Run



Paxos: Well-behaved Run



Paxos: Well-behaved Run



Paxos is Safe

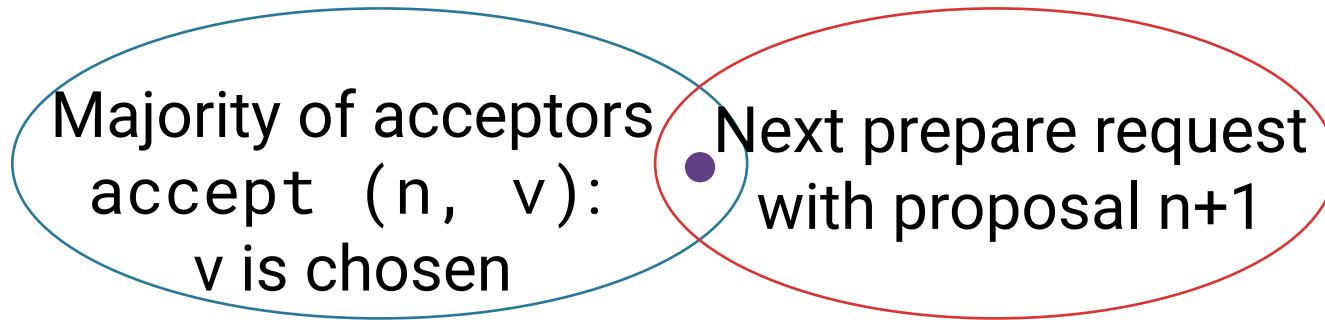


- Intuition: if proposal with value v chosen, then every higher-numbered proposal issued by any proposer has value v .

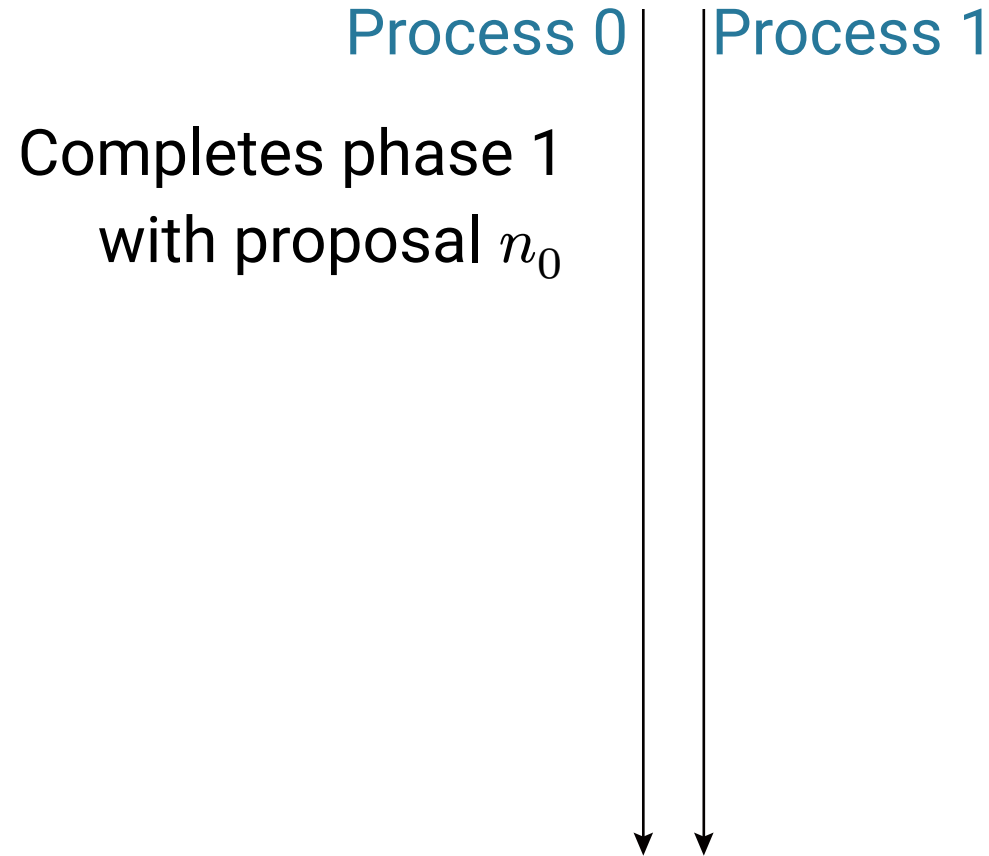
Majority of acceptors
accept (n, v) :
 v is chosen

Paxos is Safe

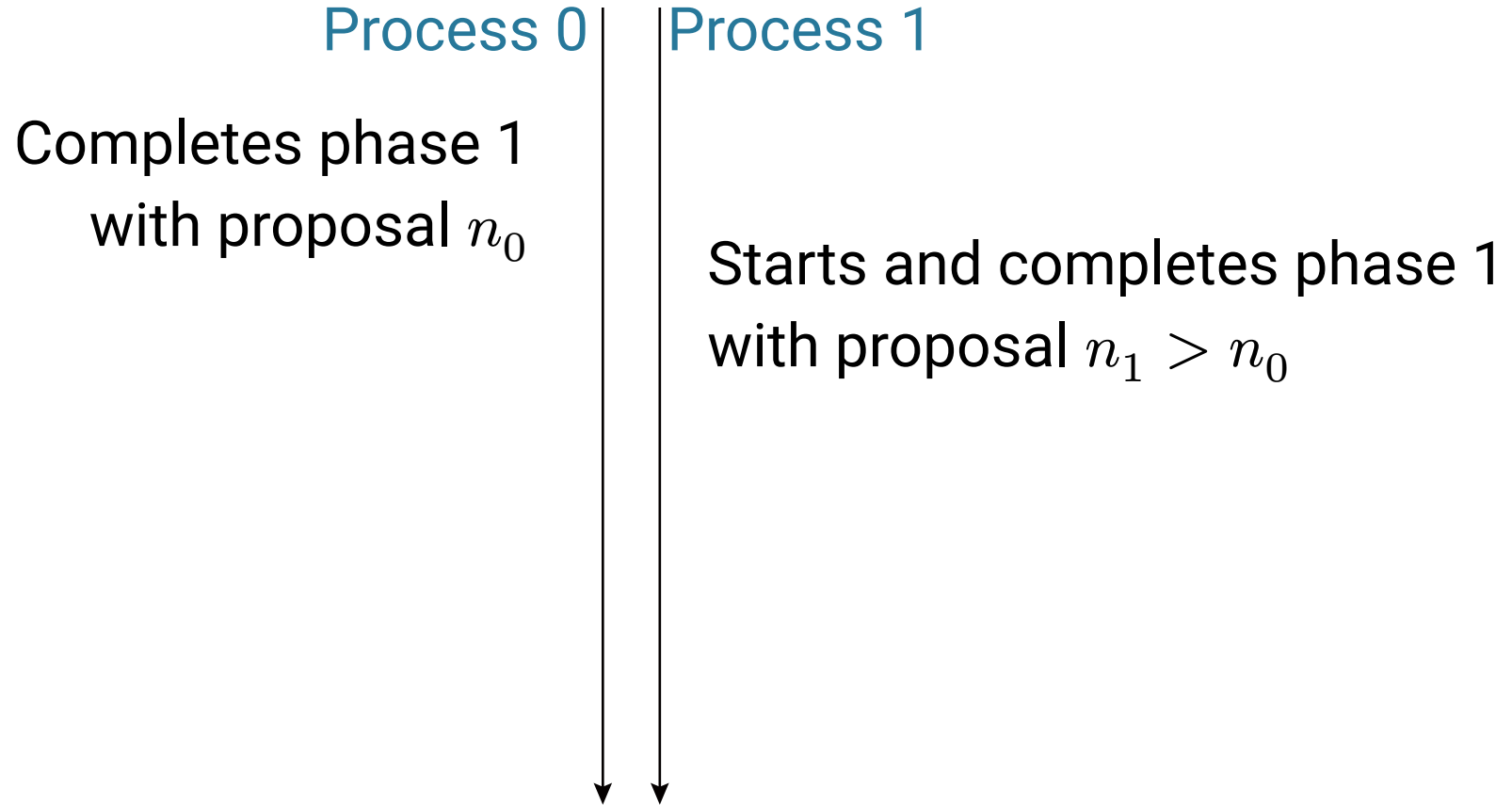
- Intuition: if proposal with value v chosen, then every higher-numbered proposal issued by any proposer has value v .



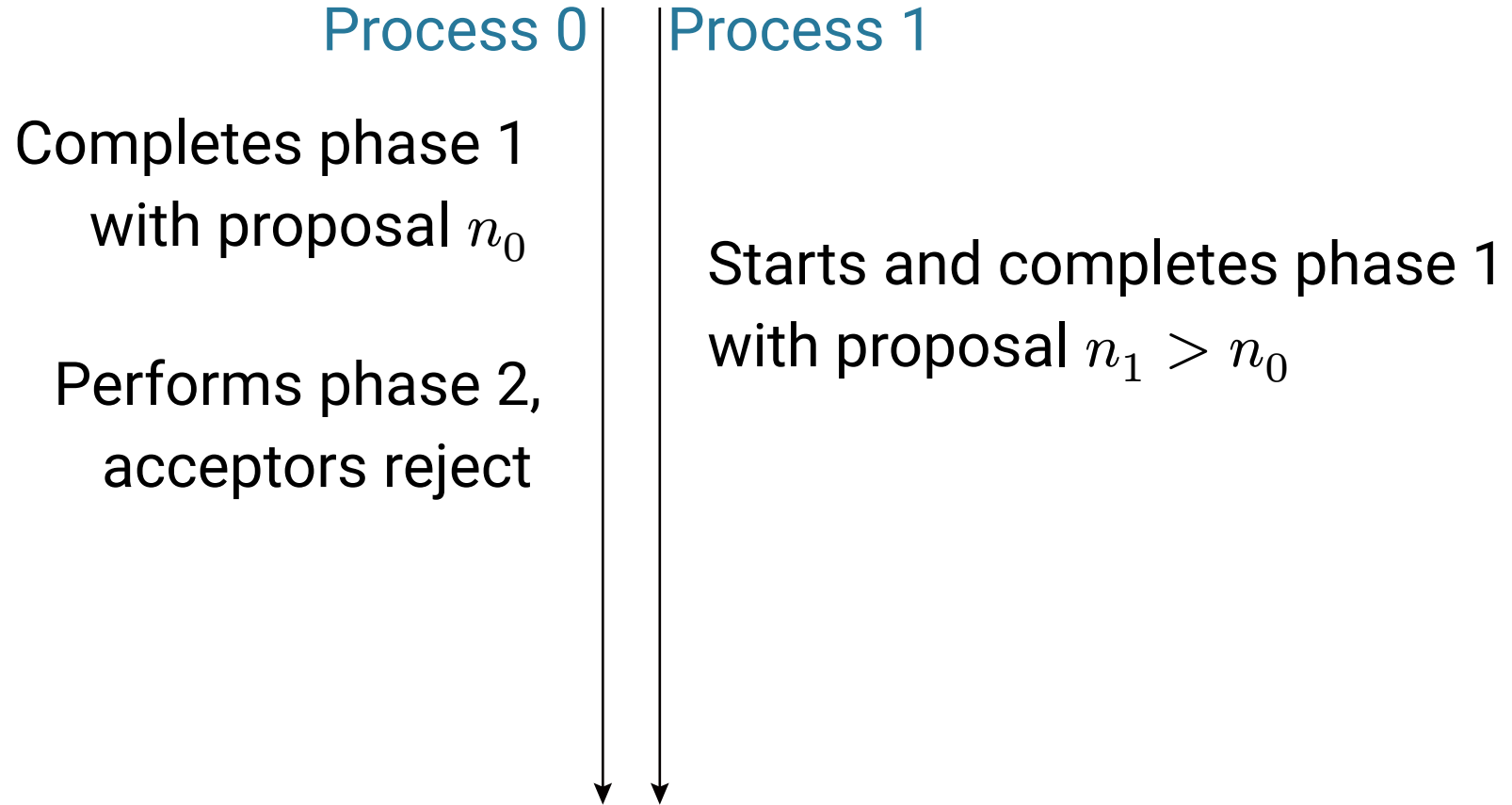
Often, But Not Always, Live



Often, But Not Always, Live



Often, But Not Always, Live



Often, But Not Always, Live



Process 0

Completes phase 1
with proposal n_0

Performs phase 2,
acceptors reject

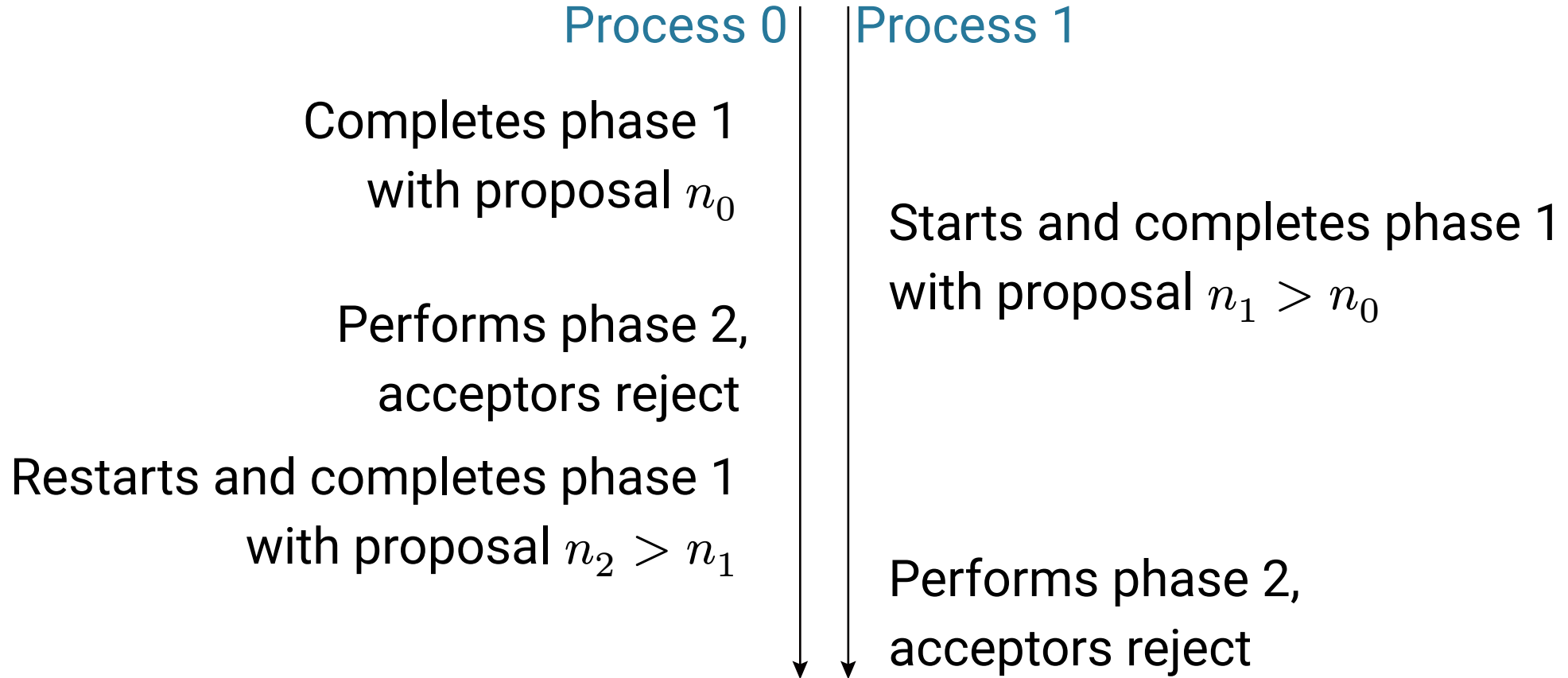
Restarts and completes phase 1
with proposal $n_2 > n_1$

Process 1

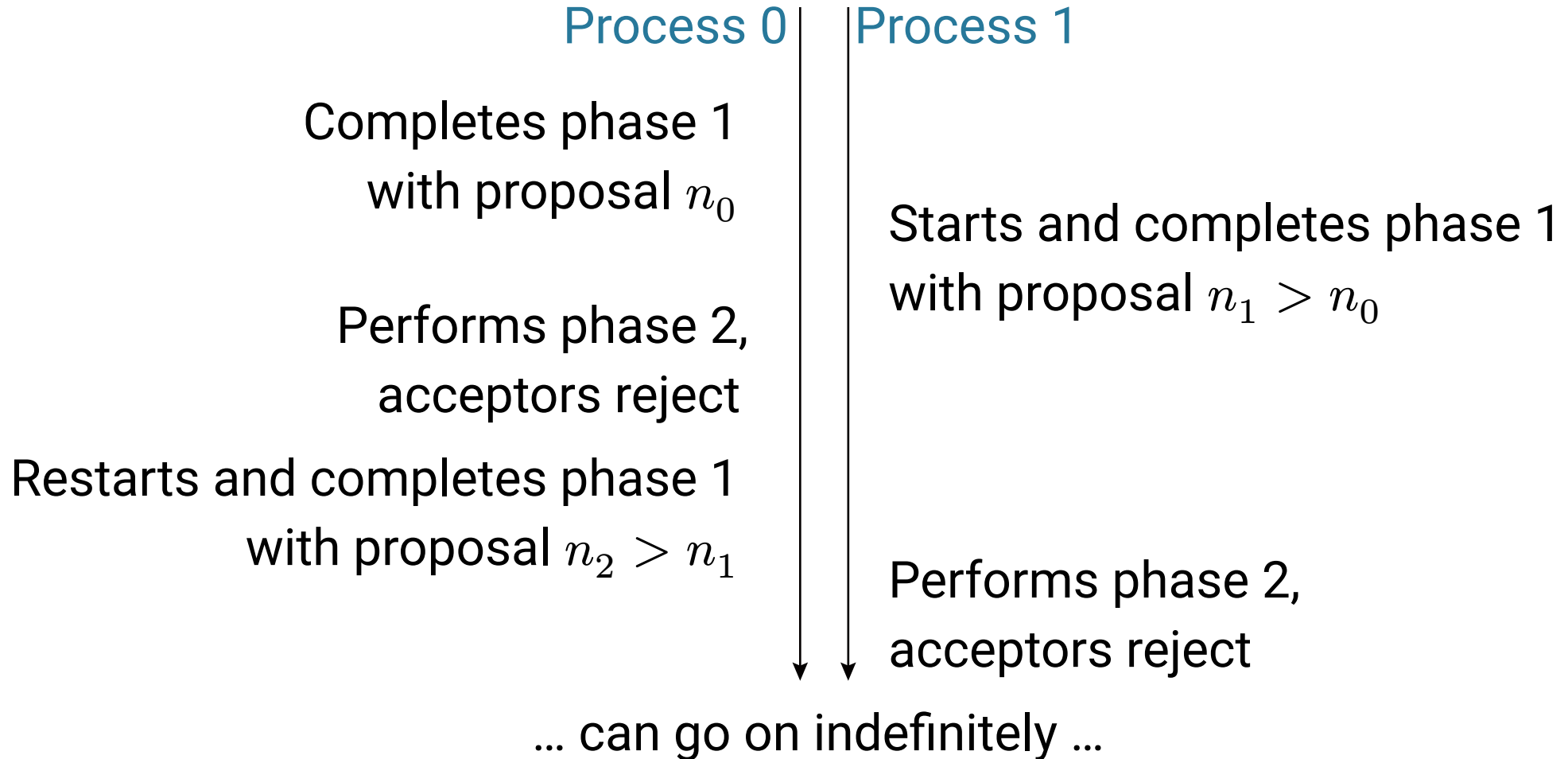
Starts and completes phase 1
with proposal $n_1 > n_0$



Often, But Not Always, Live



Often, But Not Always, Live



Paxos Summary



- Described for a single round of consensus
- Proposer, Acceptors, Learners
 - Often implemented with nodes playing all roles

Paxos Summary



- Described for a single round of consensus
- Proposer, Acceptors, Learners
 - Often implemented with nodes playing all roles
- Always safe: Quorum intersection
- Very often live

Paxos Summary



- Described for a single round of consensus
- Proposer, Acceptors, Learners
 - Often implemented with nodes playing all roles
- Always safe: Quorum intersection
- Very often live
- Acceptors accept multiple values
 - But only one value is ultimately chosen
- Once a value is accepted by a majority it is chosen

Flavors of Paxos



- Terminology is a mess
- Paxos loosely and confusingly defined...
- We'll stick with
 - Basic Paxos
 - Multi-Paxos

Flavors of Paxos: Basic Paxos

- Run the full protocol each time
 - e.g., for each slot in the command log
- Takes 2 rounds until a value is chosen

Flavors of Paxos: Multi-Paxos

- Elect a leader and have them run 2nd phase directly
 - e.g., for each slot in the command log
 - Leader election uses Basic Paxos
- Takes 1 round until a value is chosen
 - Faster than Basic Paxos
- Used extensively in practice!
 - RAFT is similar to Multi Paxos



© 2026 James Jianqiao Yu
Harbin Institute of Technology, Shenzhen

All rights reserved.

These lecture materials are intended solely for educational use in the course *Distributed Systems*. Permission is granted to students enrolled in the course to reproduce these materials, in whole or in part, for personal, non-commercial use only (including printing and photocopying). Any further reproduction or distribution, including but not limited to public posting on websites, social media, forums, or file-sharing platforms, distribution to non-enrolled individuals, or any commercial use, is prohibited without the prior written permission of the author.

This slide deck is adapted from Princeton University's *Distributed Systems* course materials, originally created by Michael Freedman, Kyle Jamieson, and Wyatt Lloyd, and licensed under the Creative Commons Attribution-NonCommercial-ShareAlike 3.0 Unported (CC BY-NC-SA 3.0) License. The original authors and source are gratefully acknowledged.